Why Inheritance Fails (UML-1 Reference)

> *“Now, let us understand why the naive inheritance-based design fails, by looking at UML-1.”*

❌ Single Responsibility Principle (SRP) — Broken

Definition (one line):

> *A class should have only one reason to change.*

What goes wrong (UML-1 reference):

> “If you look at UML-1, you can see that one player class is doing many things at the same time.
> It is playing the video, deciding video quality, handling network logic, and also managing battery conditions.”

📌 **Result:**

> “This means one class has too many responsibilities, which makes it large, complex, and difficult to maintain.”

❌ Open / Closed Principle (OCP) — Broken

Definition (one line):

> *Software should be open for extension but closed for modification.*

What goes wrong (UML-1 reference):

> “In UML-1, whenever we add a new feature like low battery mode or peak hour optimization, we must modify existing classes or create many new subclasses.”

📌 **Result:**

> “This makes the system unstable because old, working code keeps changing.”

❌ Liskov Substitution Principle (LSP) — Also Broken

Definition (one line):

> *A child class should be able to replace its parent class without breaking behavior.*

What goes wrong:

> “Some subclasses promise behavior they cannot actually support, such as 4K playback on devices that do not support it.”

📌 **Result:**

> “Replacing a parent with a child class causes unexpected behavior.”

❌ Dependency Inversion Principle (DIP) — Broken

Definition (one line):

> *High-level modules should depend on abstractions, not concrete classes.*

What goes wrong:

> “In UML-1, the Netflix player depends directly on concrete subclasses instead of abstract behaviors.”

📌 **Result:**

> “This makes testing, scaling, and future changes very difficult.”

🔴 Class Explosion Problem (UML-1 Reference)

> “Now look at UML-1 again.”

We have multiple dimensions:

- Device
- Quality
- Network
- Battery

- AI behavior

📌 **Result:**

> “Five factors quickly turn into 50 or even 200 classes in a real system.
> This is known as the class explosion problem.”

👉 **At this point, the need for a better design becomes very clear.**

🔴 Difficult to Add New Behavior

Example:

> “Suppose we want to add a rule:
> ‘If battery is low and it is peak hour, reduce quality further.’”

UML-1 impact:

- Multiple existing classes must be modified
- New subclasses must be added
- High risk of regression bugs

📌 **Developer pain:**

> “Developers become afraid to change the code.”

🔴 Code Duplication

UML-1 problem:

> “Methods like `getQuality()` are repeated in many subclasses.
> Network checks and battery logic are also duplicated.”

📌 **Result:**

> “The same logic exists in many places, making bugs harder to fix.”

🔴 Cannot Change Behavior at Runtime

```
NetflixPlayer p = new AIRecommendedHDMobilePlayer();
```

> “If the user enters a tunnel or the battery becomes low, this object cannot change its behavior.”

📌 **Result:**

> “Netflix needs mid-stream switching, but inheritance locks behavior at creation time.”

🧠 Real Developer Pain Points

> “Debugging becomes difficult because logic is spread across many subclasses.”

> “Feature rollout becomes slow because every small change affects many files.”

> “The code becomes hard to understand due to deep inheritance trees.”

> “Testing becomes a nightmare because changes create unexpected side effects.”

🧠 Short Summary (Inheritance)

> *“Inheritance becomes rigid as the system grows, while Netflix-like systems require flexibility and runtime adaptability.”*

📐 Complex UML Showing the Problem (Inheritance Explosion)

```
And one diaghram
```